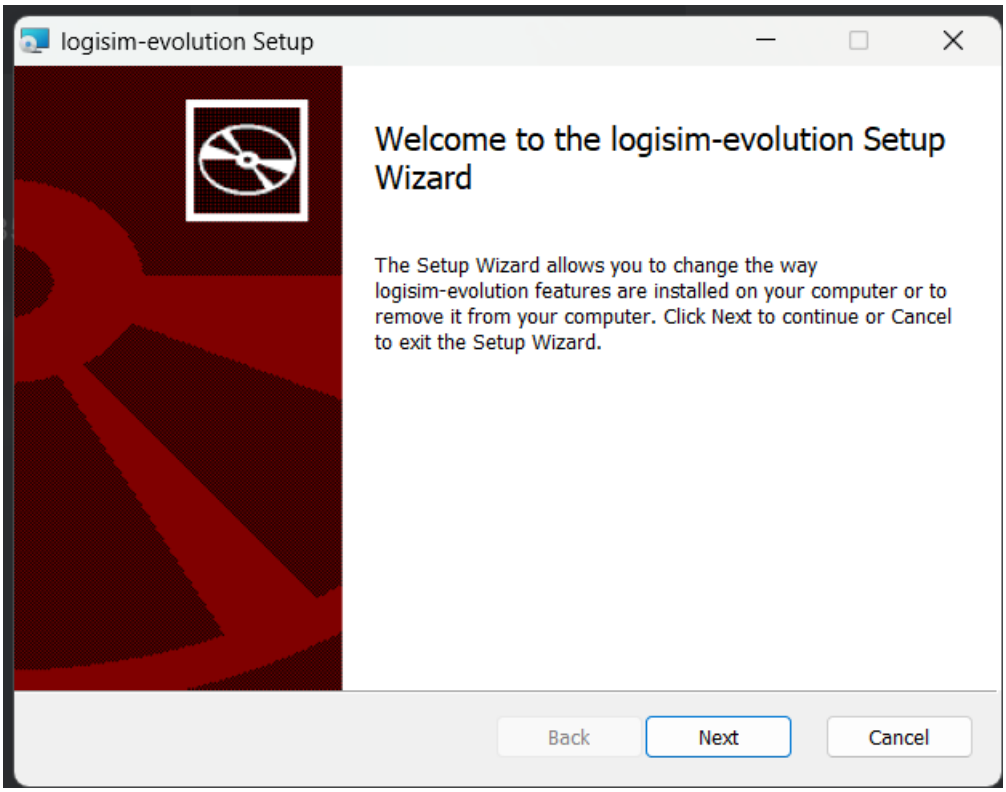
Lab 1: Introduction to Digital Logic

Software Installation Process

Before performing the digital logic experiments, the Logisim-Evolution software was installed on the computer. The following steps were followed to complete the installation.

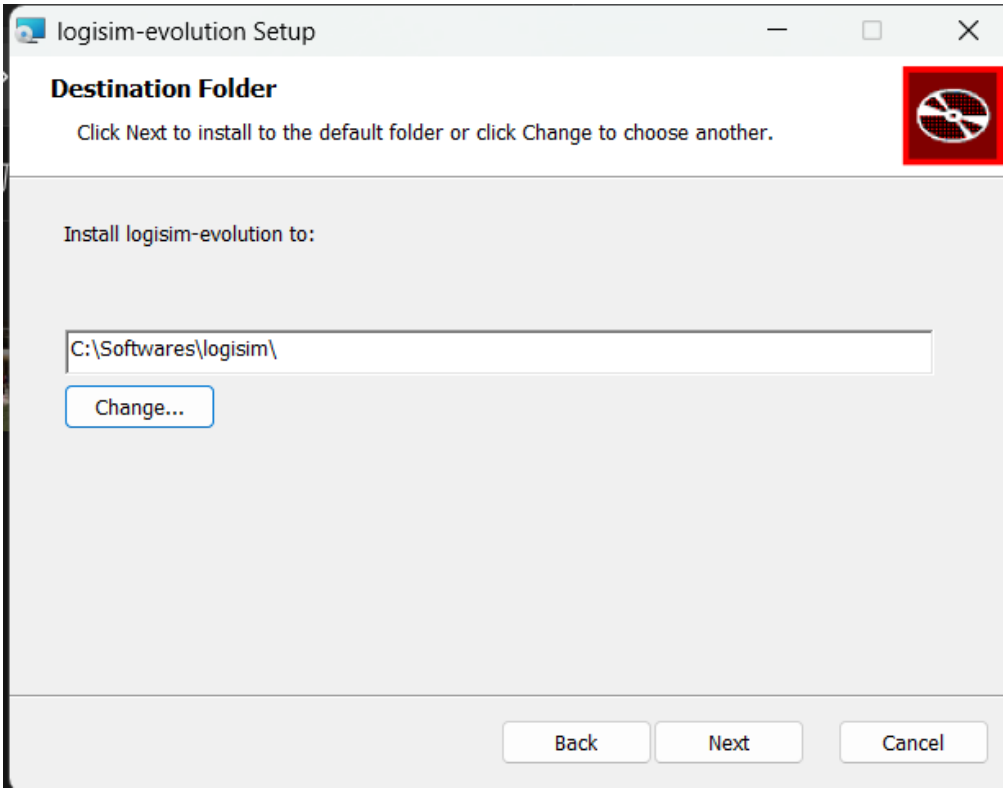
Step 1: Launch the Installer

Run the Logisim-Evolution setup file. The setup wizard opens with the welcome screen. Click the **Next** button to continue.



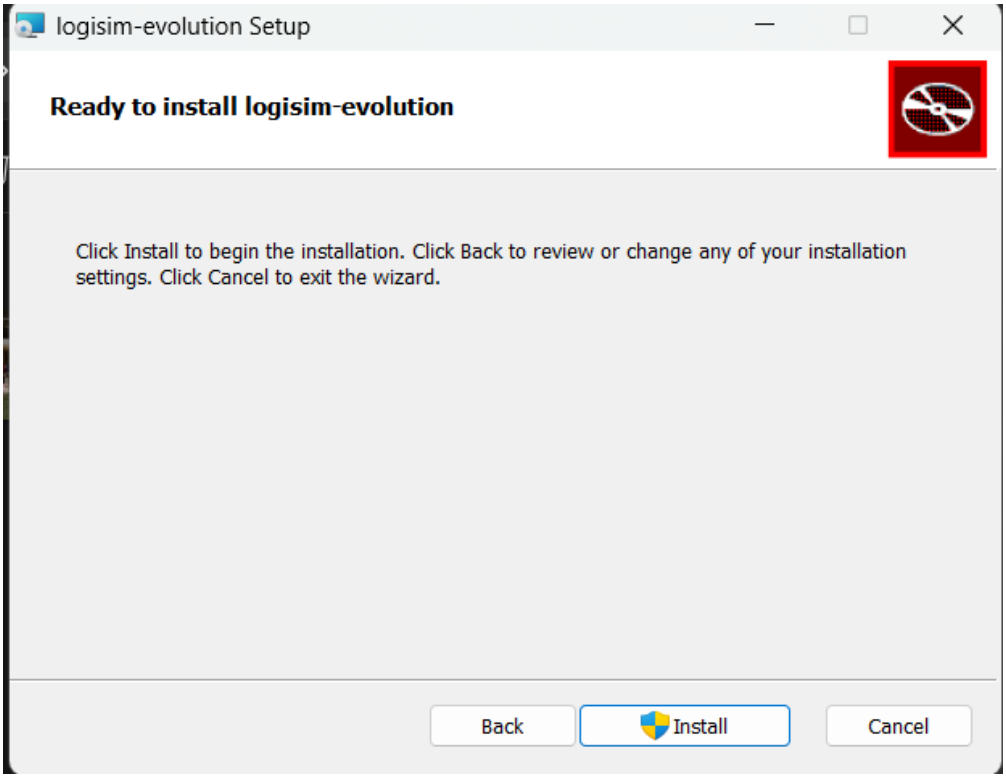
Step 2: Select the Installation Folder

Choose the destination folder where Logisim-Evolution will be installed. The default installation directory was accepted. Click **Next** to proceed.



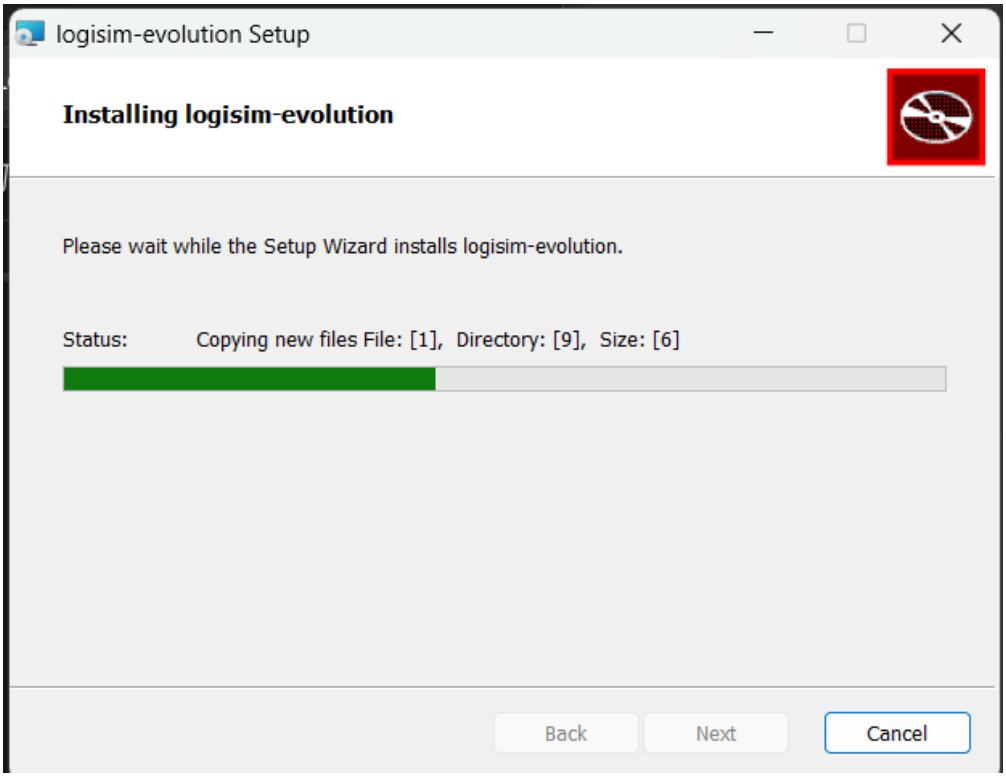
Step 3: Confirm Installation

The setup wizard displays a confirmation page. Click the **Install** button to begin the installation process.



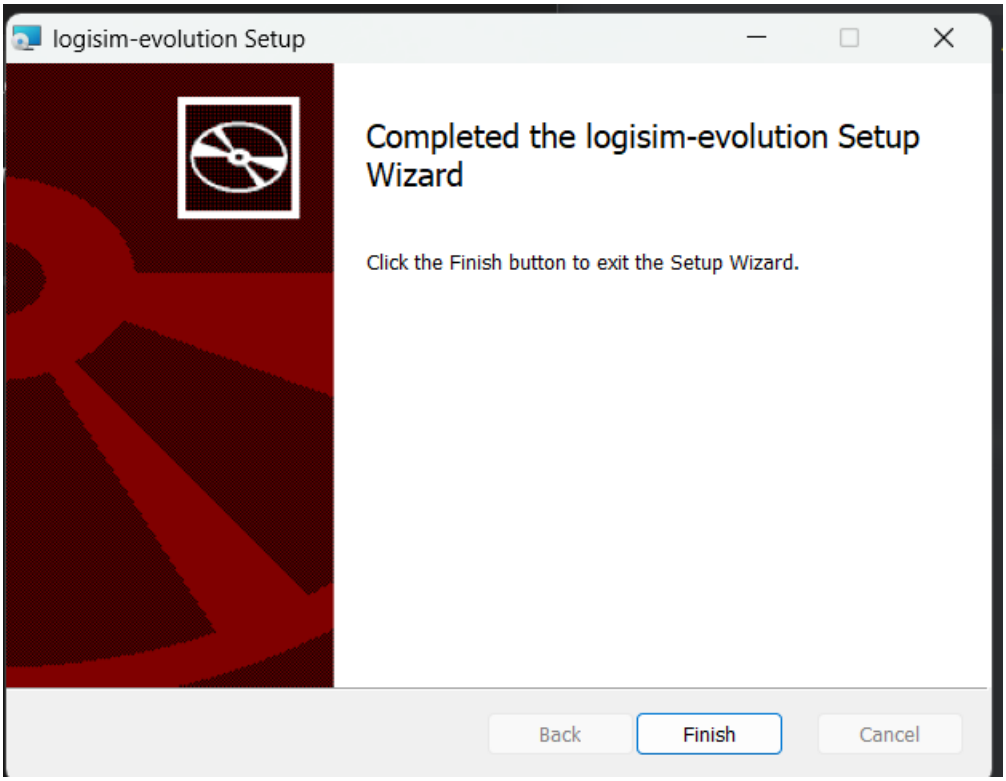
Step 4: Install the Software

The installer copies all necessary program files to the selected directory. Wait until the progress bar reaches 100%.



Step 5: Complete the Installation

After the installation finishes successfully, click the **Finish** button to close the setup wizard.



Conclusion

Logisim-Evolution was installed successfully and is ready for designing, simulating, and testing digital logic circuits used in this laboratory.

Experiment 1: Building an OR Gate from NOR Gates

Overview

This experiment focuses on constructing an OR gate using nothing but NOR gates. An OR gate is one of the basic building blocks of digital logic, producing a logical true (1) whenever at least one input is true. The NOR gate is classified as a universal gate, meaning it can be wired together to reproduce any other logic gate, OR included.

Procedure

1. The NOR Gate. A NOR gate can be thought of as an OR gate whose output is passed through a NOT gate. It only produces a 1 when every input is 0. The truth table below summarizes its behavior for two inputs.

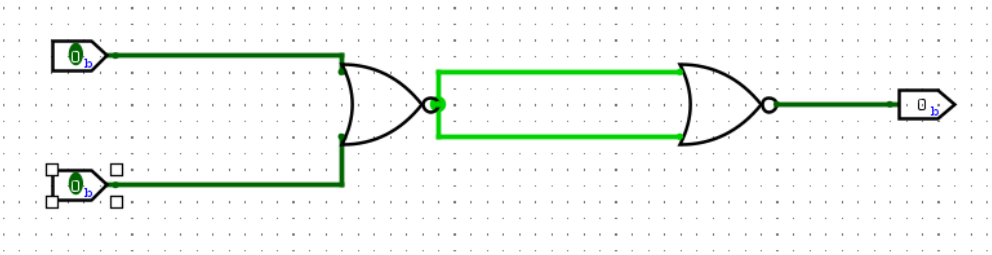
Input A	Input B	Output (A NOR B)
0	0	1
0	1	0
1	0	0
1	1	0

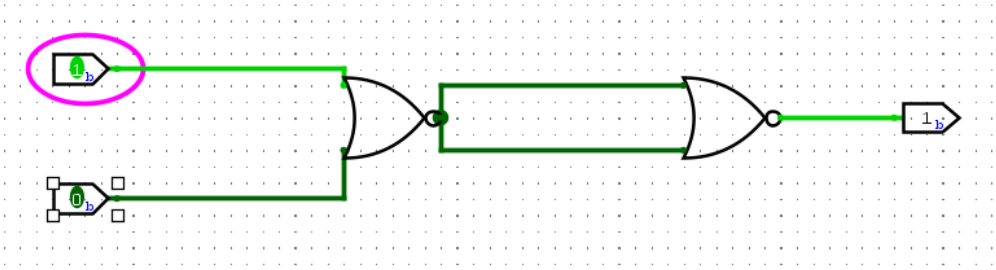
2. Deriving the OR Gate. An OR gate can be produced from NOR gates using the relation:

$$\mathrm{OR}(A, B) = \mathrm{NOT}(\mathrm{NOR}(A, B))$$

In practice, the first NOR gate computes NOR(A, B), and a second NOR gate is used purely as an inverter to flip that result back to the OR output.

3. Circuit Design. Feed inputs A and B into the first NOR gate.





Route the output of the first NOR gate into both inputs of a second NOR gate, which acts as an inverter.

4. Testing. Cycle through all combinations of A and B and confirm the circuit's output lines up with a standard OR gate.

Conclusion

The OR gate was successfully realized using only NOR gates, confirming the universal nature of the NOR gate and its capacity to substitute for other fundamental gates in digital design.

Experiment 2: Building an OR Gate from NAND Gates

Overview

Here the goal is to build an OR gate using only NAND gates, another gate type classified as universal.

Procedure

1. The NAND Gate. A NAND gate behaves like an AND gate followed by an inverter, producing a 0 only when all inputs are 1.

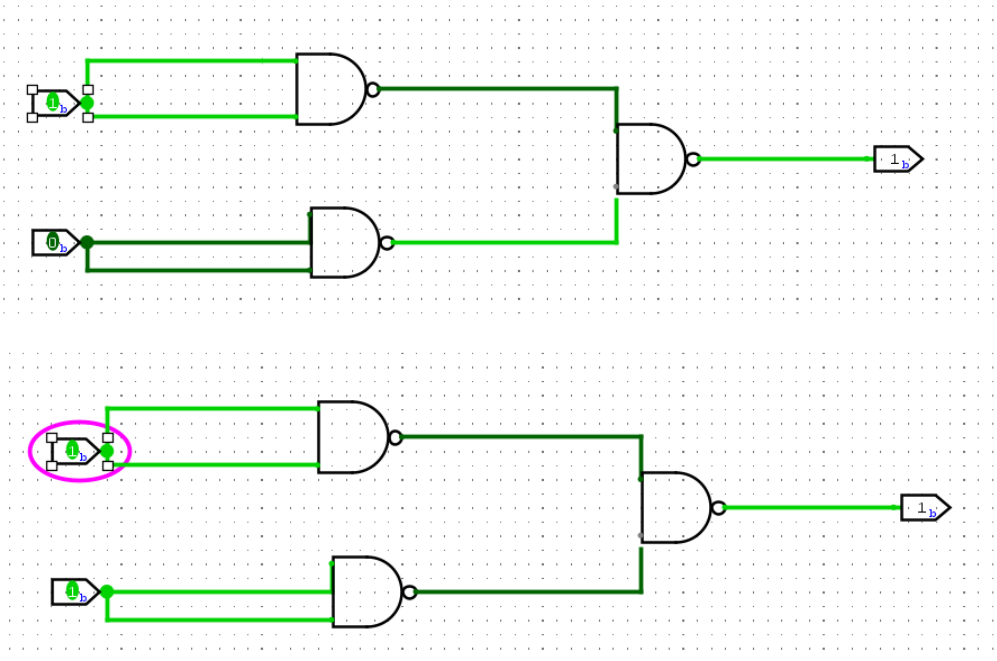
Input A	Input B	Output (A NAND B)
0	0	1
0	1	1
1	0	1
1	1	0

2. Deriving the OR Gate. Using NAND gates, the OR function is obtained from:

$$\text{OR}(A, B) = \text{NOT}(\text{NAND}(\text{NOT}(A), \text{NOT}(B)))$$

Two NAND gates first invert A and B individually, and a third NAND gate combines the inverted signals to yield the OR output.

3. Circuit Design. Wire A and B each into a separate NAND gate configured as an inverter.



4. Testing. Apply every input combination and check the output against the expected OR truth table.

Conclusion

The circuit correctly reproduced OR-gate behavior, further illustrating how the NAND gate can be reconfigured to emulate other logic functions.

Experiment 3: Building an AND Gate from NOR Gates

Overview

This experiment implements an AND gate using only NOR gates. An AND gate outputs 1 only when both of its inputs are 1.

Procedure

1. The AND Gate.

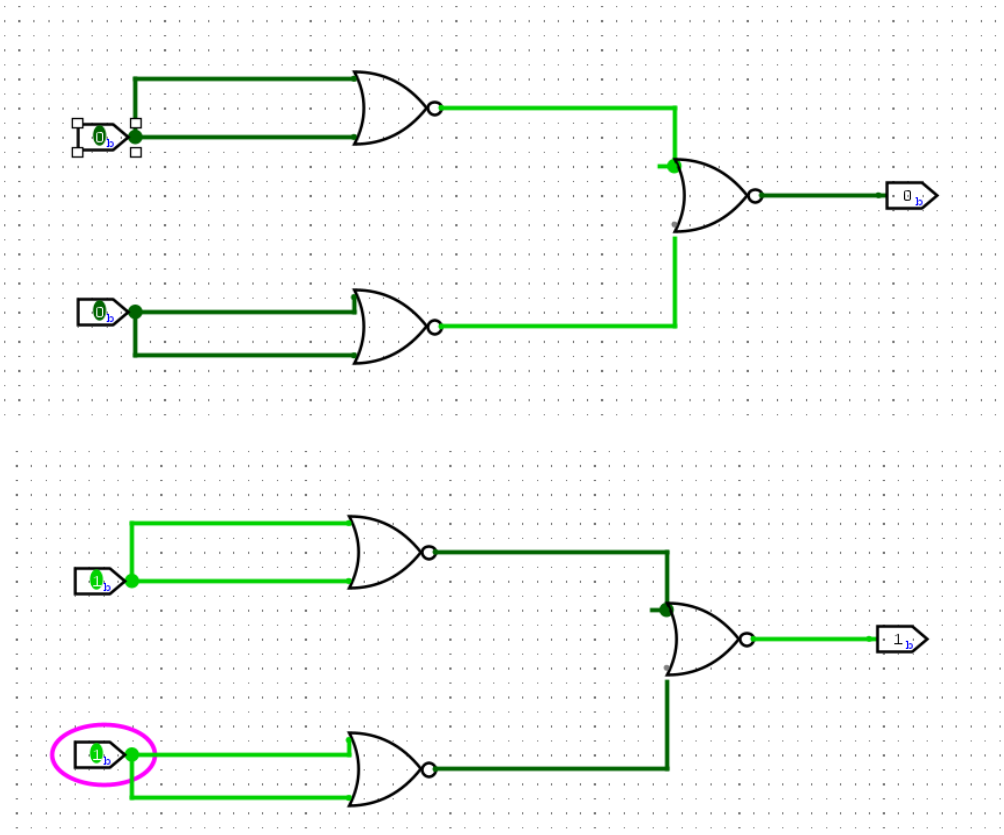
Input A	Input B	Output (A AND B)
0	0	0
0	1	0
1	0	0
1	1	1

2. Deriving the AND Gate.

$$\text{AND}(A, B) = \text{NOT}(\text{NOR}(\text{NOT}(A), \text{NOT}(B)))$$

Two NOR gates are first used to invert A and B, and a third NOR gate combines these inverted values into the final AND output.

3. Circuit Design. Invert A and B using two separate NOR gates.



4. Testing. Verify all four input combinations against the AND truth table.

Conclusion

The AND gate was successfully built from NOR gates alone, reinforcing the versatility of universal gates in digital circuit design.

Experiment 4: Building an AND Gate from NAND Gates

Overview

This experiment implements an AND gate using only NAND gates.

Procedure

1. The AND Gate.

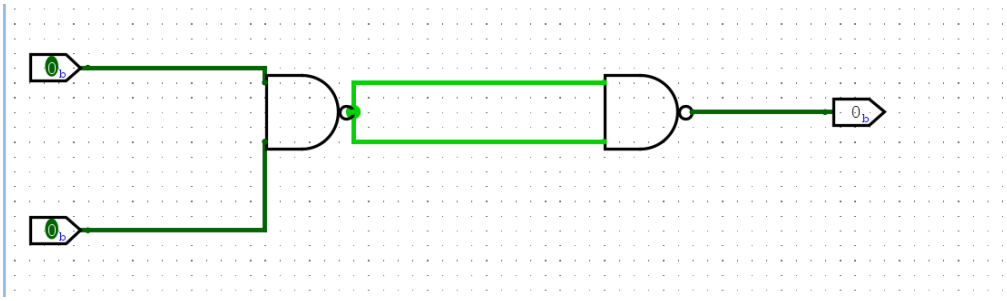
Input A	Input B	Output (A AND B)
0	0	0
0	1	0
1	0	0
1	1	1

2. Deriving the AND Gate.

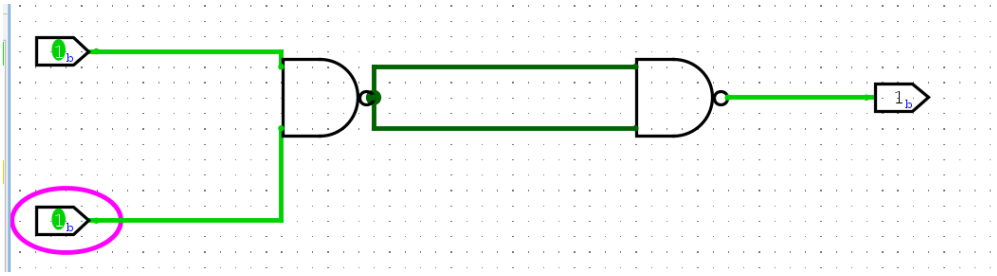
$$\text{AND}(A, B) = \text{NOT}(\text{NAND}(A, B))$$

A single NAND gate computes NAND(A, B), and a second NAND gate wired as an inverter flips this into the AND output.

3. Circuit Design. Connect A and B to the first NAND gate.



Feed its output into both inputs of a second NAND gate to invert it.



4. Testing. Check every combination of A and B against the AND truth table.

Conclusion

The resulting circuit matched the expected AND behavior, again demonstrating that a single universal gate type is sufficient to build any logic function.

Experiment 5: Building a NOT Gate from NOR Gates

Overview

This experiment produces a NOT gate using only a NOR gate. A NOT gate simply inverts its single input.

Procedure

1. The NOT Gate.

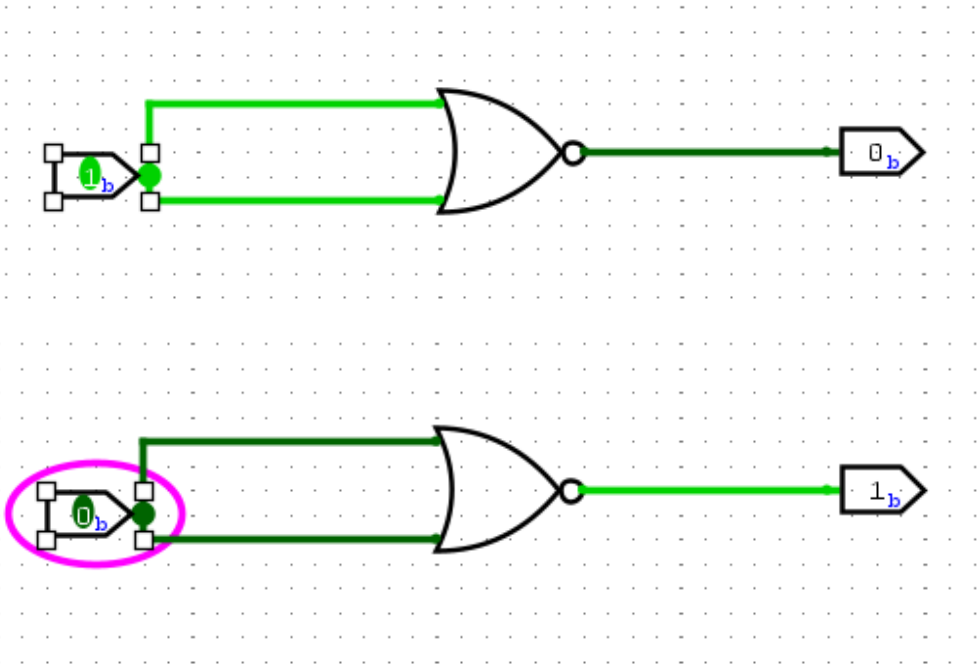
Input A	Output (NOT A)
0	1
1	0

2. Deriving the NOT Gate.

$$\text{NOT}(A) = \text{NOR}(A, A)$$

Tying both inputs of a NOR gate to the same signal A yields an inverted version of A.

3. Circuit Design. Tie input A to both inputs of a single NOR gate.



4. Testing. Apply A = 0 and A = 1 and confirm the output is inverted in each case.

Conclusion

A functioning NOT gate was obtained from a single NOR gate, confirming the expected inverter behavior.

Experiment 6: Building a NOT Gate from NAND Gates

Overview

This experiment produces a NOT gate using only a NAND gate.

Procedure

1. The NOT Gate.

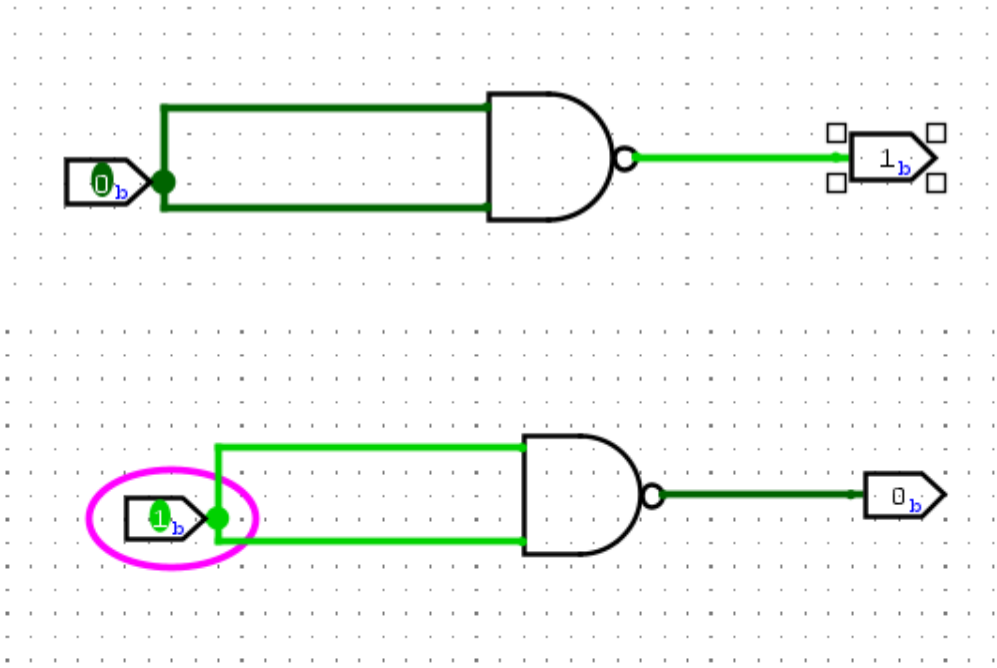
Input A	Output (NOT A)
0	1
1	0

2. Deriving the NOT Gate.

$$\text{NOT}(A) = \text{NAND}(A, A)$$

Connecting both inputs of a NAND gate to the same signal produces an inverter.

3. Circuit Design. Tie input A to both inputs of a single NAND gate.



4. Testing. Apply A = 0 and A = 1 and confirm correct inversion.

[Image Placeholder: Simulation result for NOT gate]

Conclusion

The NAND-based inverter behaved exactly as a standard NOT gate would, confirming the design.

Experiment 7: Full Adder Design and Testing

Overview

This experiment builds and tests a Full Adder circuit, a component that adds three binary bits – two operand bits and a carry-in bit – producing a sum bit and a carry-out bit.

Procedure

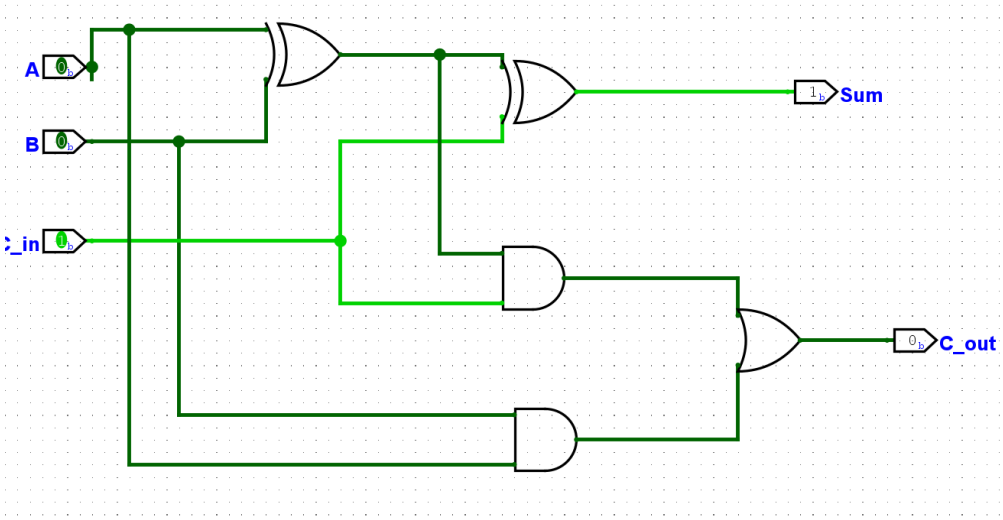
1. Full Adder Behavior. The Full Adder accepts three inputs (A, B, C_{in}) and generates two outputs (Sum, C_{out}), as shown below.

Input A	Input B	C_{in}	Sum	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

2. Building the Full Adder. A Full Adder can be assembled from two Half Adders plus an OR gate:

- The first Half Adder combines A and B, producing a partial sum S_1 and carry C_1.
- The second Half Adder combines S_1 with C_{in}, producing the final Sum and a second carry C_2.
- The final carry-out C_{out} is obtained by ORing C_1 and C_2.

3. Circuit Design. Feed A and B into the first Half Adder.



4. Testing. Run the circuit through all eight input combinations and confirm Sum and C_{out} match the truth table.

For an 8-bit test run, the overall adder reported a sum display of 10 with a carry-out of 0. Reduced to a single bit, this corresponds to Sum = 0 and $C_{out} = 1$, matching the circuit's expected behavior.

Similarly, another run reported a sum display of 11 with carry-out 0, corresponding at the single-bit level to Sum = 1 and $C_{out} = 1$, again consistent with the design.

Conclusion

The Full Adder circuit, built from two Half Adders and an OR gate, correctly performed three-bit binary addition. This building block is central to arithmetic logic units (ALUs) and other systems requiring binary addition.

Experiment 8: Binary-to-BCD Converter

Overview

This experiment implements a circuit that converts a 4-bit binary number into its Binary-Coded Decimal (BCD) equivalent, where each decimal digit is represented by its own binary code.

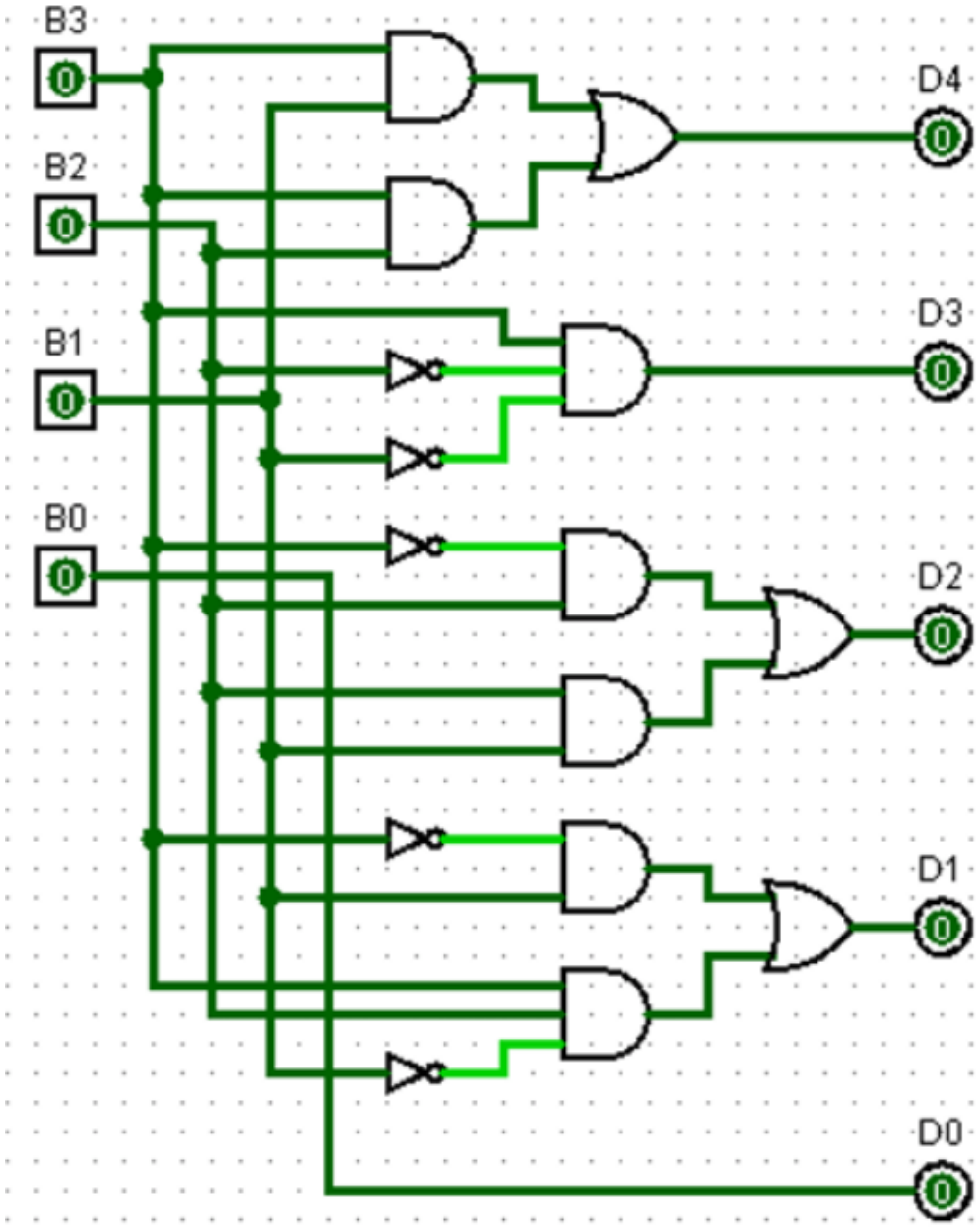
Procedure

1. Conversion Concept. A Binary-to-BCD converter accepts a binary value and outputs the corresponding BCD digits. For instance, binary 1010 (decimal 10) becomes 0001 0000 in BCD (digits "1" and "0"). The mapping for all 4-bit inputs is given below.

Binary Code				BCD Code			
B_3	B_2	B_1	B_0	D_4	D_3	D_2	D_1
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	0
0	0	1	1	0	0	1	1
0	1	0	0	0	1	0	0
0	1	0	1	0	1	0	1
0	1	1	0	0	1	1	0
0	1	1	1	0	1	1	1
1	0	0	0	1	0	0	0
1	0	0	1	1	0	0	1
1	0	1	0	1	0	1	0
1	0	1	1	1	0	1	1
1	1	0	0	1	1	0	0
1	1	0	1	1	1	0	1
1	1	1	0	1	1	1	0
1	1	1	1	1	1	1	1

2. Building the Converter. The converter is realized as a network of logic gates mapping the four binary inputs to logic expressions for each BCD output bit (D_4, D_3, D_2, D_1), which can then be driven onto a seven-segment display.

3. Circuit Design. Wire the binary inputs B_3, B_2, B_1, B_0 into the converter circuit.



The converter produces the BCD outputs D_4, D_3, D_2, D_1, which feed a BCD-to-seven-segment display.

4. Testing. Sweep through binary input combinations and confirm the BCD outputs match the expected table.

Conclusion

The Binary-to-BCD converter correctly translated binary values into their BCD equivalents, demonstrating a practical application of digital number-system conversion in devices such as calculators and digital displays.

End of Lab 1 Report